Guía · imagen de GoldenBridge con smart-turn en punto flotante (fp32)

Del primer comando a llamadas de prueba con todo el stack. Cada paso dice **dónde** se ejecuta:

- **WSL** = la terminal de Ubuntu en la laptop (Ryzen 5 7535HS, **sin** `avx512_vnni`). Usa el Docker propio de WSL, no Docker Desktop.
- **.143** = el servidor de producción (c7i.4xlarge, **con** `avx512_vnni`), por SSH.

paso	dónde	qué	duración
0	.143	leer el umbral y la ruta del modelo de producción	1 min
1	WSL	prueba de decisiones, mitad sin VNNI	~10 min
2	.143	prueba de decisiones, mitad con VNNI	~10 min
3	WSL	juntar los dos resultados y decidir	2 min
—		**si el paso 3 no da `salida=0`, se para aquí**	
4	WSL	preparar el código de GoldenBridge	5 min
5	WSL	construir la imagen, regenerar la referencia, reconstruir	~20 min
6	WSL	comprobar el guardián de arranque en una CPU sin VNNI	2 min
7	WSL	copiar la imagen a .143	~5 min
8	.143	comprobar el guardián y levantar el contenedor de prueba	5 min
9	.143	apuntar el engine al contenedor y hacer llamadas	lo que duren
10	.143	devolver el engine a producción	2 min

Los pasos 9 y 10 van **fuera de horario de campaña**, con el marcador detenido: reiniciar el engine corta las llamadas que tenga en curso.

Paso 0 · .143 · umbral y ruta del modelo de producción

```
docker run --rm --network none --entrypoint python \
  -v /srv/GoldenBridge/config/active.yaml:/in.yaml:ro goldenbridge:314t -c '
import yaml
s = yaml.safe_load(open("/in.yaml"))["smart_turn"]
for k in ("threshold", "midspeech_threshold", "model_path"):
    print(f"{k}: {s.get(k)}")'
```

Anota `threshold` (se espera **0.35**): es el `UMBRAL` del paso 4. Si `model_path` es una ruta absoluta de `/srv/...`, es normal: el paso 8 la reemplaza, porque dentro del contenedor no existe.

Si dice `Permission denied`, el archivo no es legible por otros usuarios: `sudo chmod o+r /srv/GoldenBridge/config/active.yaml` y repetir.

Paso 1 · WSL · prueba de decisiones, mitad sin VNNI

```
# 1.1 · la CPU no debe tener VNNI (las dos líneas tienen que dar 0)
grep -c avx512_vnni /proc/cpuinfo
grep -c avx_vnni /proc/cpuinfo

# 1.2 · el repo de análisis, clonado desde la copia de Windows (sin credenciales)
git clone /mnt/c/Users/GESTION/Documents/proyectos/refGolden ~/goldenbridge-analisis
cd ~/goldenbridge-analisis && git switch paso/smartturn-fp32

# 1.3 · la imagen actual en el Docker de WSL
cd /mnt/c/Users/GESTION/Documents/proyectos/refGolden/files
sha256sum -c goldenbridge-314t.tar.gz.sha256
docker load < goldenbridge-314t.tar.gz

# 1.4 · el modelo fp32 (anota el sha256: en .143 tiene que dar lo mismo)
curl -L -o "$HOME/smart-turn-v3.2-gpu.onnx" \
  https://huggingface.co/pipecat-ai/smart-turn-v3/resolve/main/smart-turn-v3.2-gpu.onnx
ls -l "$HOME/smart-turn-v3.2-gpu.onnx"      # ~32 MB
sha256sum "$HOME/smart-turn-v3.2-gpu.onnx"

# 1.5 · correr
cd ~/goldenbridge-analisis/experimentos/2026-10-14_smartturn-fp32
AUDIOS=/mnt/c/Users/GESTION/Documents/proyectos/refGolden/GoldenBridge/tools/loadtest/utterances \
MODELO_FP32="$HOME/smart-turn-v3.2-gpu.onnx" ./run.sh
```

Al final imprime `casos: ~3000 · vnni: False`. El resultado queda en `resultados/<sello>/resultado.json`.

Paso 2 · .143 · prueba de decisiones, mitad con VNNI

```
# 2.1 · la CPU tiene VNNI (tiene que dar más que 0)
grep -c avx512_vnni /proc/cpuinfo

# 2.2 · el repo (si quedó la carpeta de resultados de la prueba del guardián, borrarla antes del pull)
rm -rf ~/goldenbridge-analisis/experimentos/2026-10-09_m0-gate/resultados/20261010T052356Z
cd ~/goldenbridge-analisis && git fetch && git switch paso/smartturn-fp32 && git pull

# 2.3 · el mismo modelo fp32 (el sha256 tiene que coincidir con el del paso 1.4)
curl -L -o "$HOME/smart-turn-v3.2-gpu.onnx" \
  https://huggingface.co/pipecat-ai/smart-turn-v3/resolve/main/smart-turn-v3.2-gpu.onnx
sha256sum "$HOME/smart-turn-v3.2-gpu.onnx"

# 2.4 · correr (los audios se toman de /srv/GoldenBridge/tools/loadtest/utterances)
cd ~/goldenbridge-analisis/experimentos/2026-10-14_smartturn-fp32
MODELO_FP32="$HOME/smart-turn-v3.2-gpu.onnx" ./run.sh
```

No toca producción: corre la imagen aparte, sin red. Al final imprime `vnni: True`.

Paso 3 · WSL · juntar los resultados y decidir

```
cd ~/goldenbridge-analisis/experimentos/2026-10-14_smartturn-fp32

# 3.1 · traer el resultado de .143
REMOTO=goldenbridge-analisis/experimentos/2026-10-14_smartturn-fp32/resultados
SELLO_143=$(ssh -i ~/.ssh/ai_key_eraxiom eraxiom@16.58.43.143 "ls -1 $REMOTO | grep -v crudo | tail -1")
scp -i ~/.ssh/ai_key_eraxiom "eraxiom@16.58.43.143:$REMOTO/$SELLO_143/resultado.json" "$HOME/resultado_143.json"

# 3.2 · comparar
SELLO_LAP=$(ls -1 resultados | grep -v crudo | tail -1)
python3 analiza.py "$HOME/resultado_143.json" "resultados/$SELLO_LAP/resultado.json"; echo "salida=$?"
```

- **`salida=0`**: el fp32 decide igual en las dos CPUs. Sigue al paso 4.
- **`salida=1`**: el fp32 también se mueve entre CPUs. **Se para aquí**: el requisito de VNNI se queda. Pásame la salida.
- **`salida=2`**: corrida inválida (mismos flags, modelos distintos o stacks distintos). Revisar los avisos.

Para guardar los resultados en el repo:

```
DEST=/mnt/c/Users/GESTION/Documents/proyectos/refGolden/experimentos/2026-10-14_smartturn-fp32/resultados
mkdir -p "$DEST/$SELLO_LAP" "$DEST/$SELLO_143"
cp resultados/$SELLO_LAP/{resultado.json,manifiesto.txt,parametros.txt} "$DEST/$SELLO_LAP/"
cp "$HOME/resultado_143.json" "$DEST/$SELLO_143/resultado.json"
```

(El manifiesto de .143 lo sube la sesión del servidor con `git add … && git push`, o me lo pegas.)

Paso 4 · WSL · preparar el código de GoldenBridge

Sobre `feat/modal-migration` (de donde sale la imagen), en una copia aparte: la copia de Windows tiene trabajo local en otra rama y no se toca.

```
# 4.1 · copia de trabajo en WSL, en la base de la imagen
git clone /mnt/c/Users/GESTION/Documents/proyectos/refGolden/GoldenBridge ~/gb-fp32
cd ~/gb-fp32
git fetch origin refs/remotes/origin/feat/modal-migration:refs/heads/base-modal
git switch -c fix/smartturn-fp32 base-modal
git log -1 --format='%h %s'              # 9c024fd
git config user.name "emerson rios" && git config user.email "erios@fenixcorp.gt"

# 4.2 · el guardián de arranque corregido (cumple el contrato de Modal, exige 0 saltadas)
git am /mnt/c/Users/GESTION/Documents/proyectos/refGolden/parches/2026-10-09_goldenbridge_gate-arranque/0001-*.patch

# 4.3 · los modelos, sacados de la imagen actual (no están en el repo) + el fp32
mkdir -p modelos
docker create --name extraer goldenbridge:314t
docker cp extraer:/app/smart-turn modelos/smart-turn
docker cp extraer:/opt/modelos/torch/hub modelos/torch-hub
docker cp extraer:/opt/modelos/MANIFIESTO.txt modelos/MANIFIESTO.txt
docker rm extraer
cp "$HOME/smart-turn-v3.2-gpu.onnx" modelos/smart-turn/
ls -l modelos/smart-turn                 # los dos .onnx: 8,7 MB y 32 MB

# 4.4 · el umbral de producción (el del paso 0)
UMBRAL=0.35

# 4.5 · los cambios de código
sed -i 's|smart-turn-v3.2-cpu.onnx|smart-turn-v3.2-gpu.onnx|' \
  config/default.yaml src/processors/smart_turn_processor.py \
  test/reference/generate_reference.py docker/arranque.sh
sed -i 's/^DECIDEN = ("avx512_vnni",)$/DECIDEN = ()/' test/test_reference_features.py
sed -i "s/^THRESHOLD = 0.6$/THRESHOLD = $UMBRAL/" test/reference/generate_reference.py
sed -i 's/^\( *\)so.inter_op_num_threads = 1$/&\n\1so.intra_op_num_threads = 1/' \
  test/reference/generate_reference.py test/test_reference_features.py

# 4.6 · los casos nuevos de la referencia, elegidos sobre el fp32 en la CPU de producción
python3 ~/goldenbridge-analisis/experimentos/2026-10-14_smartturn-fp32/elige_cortes.py \
  "$HOME/resultado_143.json" --umbral "$UMBRAL" | tee "$HOME/cortes.txt"
CONF=$(grep '^CONFIDENT_MS' "$HOME/cortes.txt"); MARG=$(grep '^MARGINAL_MS' "$HOME/cortes.txt")
sed -i "s/^CONFIDENT_MS = .*/$CONF/; s/^MARGINAL_MS = .*/$MARG/" test/reference/generate_reference.py

# 4.7 · revisar que quedó todo
grep -n "gpu.onnx\|^DECIDEN\|^THRESHOLD\|^CONFIDENT_MS\|^MARGINAL_MS\|intra_op_num_threads" \
  config/default.yaml src/processors/smart_turn_processor.py docker/arranque.sh \
  test/reference/generate_reference.py test/test_reference_features.py
git diff --stat
```

En el 4.7 tienen que aparecer: `gpu.onnx` en los cuatro archivos, `DECIDEN = ()`, `THRESHOLD = 0.35`, las dos listas nuevas y `intra_op_num_threads = 1` en los dos archivos de prueba.

```
# 4.8 · guardar el cambio
git commit -a -m "smart-turn fp32: modelo sin cuantizar; el guardián deja de exigir avx512_vnni

Evidencia: goldenbridge-analisis experimentos/2026-10-14_smartturn-fp32
(fp32 sin decisiones volteadas entre Xeon 8488C con VNNI y Ryzen 7535HS sin VNNI).
Umbral de la referencia = el de producción ($UMBRAL); casos elegidos sobre fp32."
```

Paso 5 · WSL · construir, regenerar la referencia, reconstruir

```
cd ~/gb-fp32

# 5.1 · primera construcción (con la referencia vieja adentro; necesita internet)
docker build -t goldenbridge:314t-fp32 .

# 5.2 · regenerar la referencia DENTRO de la imagen nueva (mismo Python, numpy y ONNX Runtime)
#       --user: la imagen corre como uid 10001 y no podría escribir en tu carpeta
docker run --rm --network none --entrypoint python \
  --user "$(id -u):$(id -g)" -e HOME=/tmp -e GB_ORT_INTRA_OP=1 \
  -v "$PWD/test/reference:/app/test/reference" \
  goldenbridge:314t-fp32 test/reference/generate_reference.py
```

Imprime los 14 casos con su probabilidad; los marginales llevan `<- marginal`. Tienen que quedar cerca del umbral.

```
# 5.3 · guardar la referencia nueva y reconstruir con ella adentro
git add test/reference/features.npz test/reference/stack.json
git commit -m "smart-turn fp32: referencia regenerada en la imagen nueva"
docker build -t goldenbridge:314t-fp32 .
```

Paso 6 · WSL · el guardián en una CPU sin VNNI

```
docker run --rm --network none -e GB_SOLO_GATE=1 goldenbridge:314t-fp32; echo "salida=$?"
```

Tiene que terminar con `referencia OK — el modelo decide igual que en producción`, `GB_SOLO_GATE=1 — gate verificado` y `salida=0`.

Contraste (opcional): la imagen actual en esta misma CPU **no** pasa (en agosto, 2 de 14 decisiones volteadas):

```
timeout 180 docker run --rm --network none goldenbridge:314t; echo "salida=$?"   # se espera distinta de 0
```

Paso 7 · WSL · copiar la imagen a .143

```
docker save goldenbridge:314t-fp32 | gzip > "$HOME/goldenbridge-314t-fp32.tar.gz"
cd "$HOME" && sha256sum goldenbridge-314t-fp32.tar.gz > goldenbridge-314t-fp32.tar.gz.sha256
scp -i ~/.ssh/ai_key_eraxiom "$HOME/goldenbridge-314t-fp32.tar.gz" "$HOME/goldenbridge-314t-fp32.tar.gz.sha256" \
  eraxiom@16.58.43.143:/tmp/
```

Paso 8 · .143 · el guardián y el contenedor de prueba

```
# 8.1 · cargar y verificar
cd /tmp && sha256sum -c goldenbridge-314t-fp32.tar.gz.sha256 && docker load < goldenbridge-314t-fp32.tar.gz
docker run --rm --network none -e GB_SOLO_GATE=1 goldenbridge:314t-fp32; echo "salida=$?"   # salida=0

# 8.2 · la config: la de producción con la ruta del modelo DENTRO del contenedor
mkdir -p "$HOME/fp32-lab"
docker run --rm --network none --entrypoint python \
  -v /srv/GoldenBridge/config/active.yaml:/in.yaml:ro goldenbridge:314t-fp32 -c '
import sys, yaml
c = yaml.safe_load(open("/in.yaml")) or {}
c.setdefault("smart_turn", {})["model_path"] = "/app/smart-turn/smart-turn-v3.2-gpu.onnx"
yaml.safe_dump(c, sys.stdout, sort_keys=False, allow_unicode=True)' > "$HOME/fp32-lab/config.yaml"
chmod 644 "$HOME/fp32-lab/config.yaml"
grep -n "model_path" "$HOME/fp32-lab/config.yaml"
grep -n "/srv" "$HOME/fp32-lab/config.yaml" || echo "sin otras rutas de /srv"
```

Si el último `grep` muestra otras rutas de `/srv`, pásamelas antes de seguir: dentro del contenedor no existen.

```
# 8.3 · credenciales (sólo las tres de los proveedores; no se imprimen)
ENV_ARGS=()
for n in GOTENKS_API_KEY URSULA_API_KEY CEREBRAS_API_KEY; do
  l=$(grep -E "^[[:space:]]*(export[[:space:]]+)?$n=" /srv/GoldenBridge/.env | tail -1) || true
  [ -n "$l" ] || { echo "falta $n"; continue; }
  v=${l#*=}; v=${v%$'\r'}; v=${v#\"}; v=${v%\"}; v=${v#\'}; v=${v%\'}
  export "$n=$v"; ENV_ARGS+=(-e "$n")
done; unset l v
echo "credenciales pasadas: $(( ${#ENV_ARGS[@]} / 2 ))"

# 8.4 · lo que la imagen no trae y una llamada real necesita: campañas, modelos auxiliares, audio de fondo
MONTAJES=()
for d in campaigns models background_audio.wav; do
  [ -e "/srv/GoldenBridge/$d" ] && MONTAJES+=(-v "/srv/GoldenBridge/$d:/app/$d:ro")
done
echo "montajes: ${MONTAJES[*]}"

# 8.5 · levantar en el puerto 8776 (producción sigue en 8765, intacta)
docker run -d --name gb-fp32 -p 127.0.0.1:8776:8776 \
  -e GB_PUERTO=8776 -e GB_CONFIG=/lab/config.yaml \
  -v "$HOME/fp32-lab/config.yaml:/lab/config.yaml:ro" \
  "${MONTAJES[@]}" "${ENV_ARGS[@]}" goldenbridge:314t-fp32

# 8.6 · comprobar (esperar ~40 s: el guardián corre primero)
sleep 40
docker logs gb-fp32 2>&1 | grep -E "referencia OK|Smart Turn processor created|not available|Error|Traceback"
curl -s http://127.0.0.1:8776/health; echo
```

Tiene que aparecer `referencia OK`, `Smart Turn processor created … model_path=/app/smart-turn/smart-turn-v3.2-gpu.onnx`, ningún `not available` y `"status":"healthy"`.

Paso 9 · .143 · llamadas de prueba con todo el stack

Fuera de horario de campaña y con el marcador detenido.

```
# 9.1 · dónde está desplegado el engine y a qué GoldenBridge apunta hoy (ANOTAR el valor)
DIR_ENGINE=$(docker inspect ai_engine --format '{{ index .Config.Labels "com.docker.compose.project.working_dir" }}')
echo "$DIR_ENGINE"
docker exec ai_engine printenv GOLDEN_WS_URL
cd "$DIR_ENGINE" && docker compose config | grep -n GOLDEN_WS_URL
grep -n "^GOLDEN_WS_URL" .env
```

Si `GOLDEN_WS_URL` **no** está en `.env` (el último `grep` no imprime nada), para aquí y pásame la salida de `docker compose config | grep -n GOLDEN_WS_URL`: está definido en otro lado.

```
# 9.2 · apuntar el engine al contenedor de prueba
cp .env .env.antes-fp32
sed -i 's|^GOLDEN_WS_URL=.*|GOLDEN_WS_URL=ws://127.0.0.1:8776/v1/agent/converse|' .env
docker compose up -d ai_engine
docker exec ai_engine printenv GOLDEN_WS_URL        # tiene que decir ...:8776/...

# 9.3 · seguir el contenedor mientras haces las llamadas (Ctrl+C para salir)
docker logs -f --since 1m gb-fp32 2>&1 | grep -E "Per-call variables|Smart Turn|turn|WebSocket connection closed|Error"
```

Haz las llamadas de prueba como siempre (extensión o campaña de prueba). Lo que hay que confirmar: que el bot detecta el fin de turno a tiempo, que no corta a mitad de frase y que las interrupciones funcionan.

Paso 10 · .143 · devolver el engine a producción y limpiar

```
cd "$DIR_ENGINE"
cp .env.antes-fp32 .env
docker compose up -d ai_engine
docker exec ai_engine printenv GOLDEN_WS_URL        # el valor anotado en 9.1

docker logs --timestamps gb-fp32 > "$HOME/fp32-lab/gb-fp32_$(date -u +%Y%m%dT%H%M%SZ).log" 2>&1
docker rm -f gb-fp32
```

El log queda en `~/fp32-lab/`. Tiene datos de las llamadas: **no va al repo**. Si quieres que lo analice, pásamelo por otro medio.

Lo que esta guía no cubre

- **Llamadas completas sobre una CPU sin VNNI.** El paso 6 prueba el guardián sin VNNI y el paso 9 las llamadas con VNNI. Para llamadas sobre una CPU sin VNNI hace falta una EC2 c6a.
- **El código de F261.** La imagen sale de `feat/modal-migration` (= `api`), sin los 24 commits de `feat/f261-call-summary` que corren en producción. Las llamadas de prueba no tendrán esas funciones.
- **Si el umbral 0,35 sigue siendo el correcto con el fp32.** Se ajustó con el modelo cuantizado. El paso 3 lista las decisiones que cambian entre los dos modelos; las llamadas del paso 9 dicen si el bot se comporta bien.